

Course Scheduling via Graph Coloring: A Coursework Case Study Using Greedy Heuristics in Python

Kaustubha Venkata Eluri
Northeastern University
Email: eluri.k@northeastern.edu

Abstract—This paper documents a course project completed for CS 5002 (Discrete Structures) at Northeastern University in Spring 2024 by a five-member team, modeling university course scheduling as a graph coloring problem. Each course is represented as a vertex, and an edge is added between two courses whenever at least one student is enrolled in both, so that a proper coloring of the resulting conflict graph corresponds to a conflict-free assignment of courses to time slots. The team implemented a greedy vertex-coloring heuristic in Python using the NetworkX library and validated it against NetworkX’s built-in `greedy_color` routine on a small, hand-constructed example of 8 students and 7 courses. The greedy algorithm produced a valid 3-coloring on this example, matching the result of NetworkX’s own greedy coloring implementation. We present the graph model, the greedy algorithm, and the honestly-scoped results and limitations documented in the team’s original project report and README, including the explicit acknowledgment that a Welsh–Powell (degree-ordered) heuristic was considered as a possible improvement but was not implemented. This is a small-scale coursework artifact, not a peer-reviewed research contribution, and its results should be read accordingly.

Index Terms—graph coloring, course scheduling, greedy algorithm, NetworkX, discrete structures, coursework project

I. INTRODUCTION

Universities must assign each offered course to one of a limited number of time slots such that no student is required to attend two courses scheduled at the same time. As the number of courses and cross-enrollments grows, manually finding a conflict-free schedule with few time slots becomes difficult. This is a classic instance of the graph coloring problem, in which the objects to be scheduled are represented as vertices, pairwise conflicts are represented as edges, and a valid schedule corresponds to a proper coloring of the graph in which no two adjacent vertices share a color.

This paper reports on a group project completed for the course CS 5002 (Discrete Structures) at Northeastern University in Spring 2024 by a team of five students: Yao Chong (Charles) Chow, Kaustubha Eluri, Yuneng (James) Li, Jonathan Mowat, and Nicholas Ung. The team’s motivation, as documented in the project report, arose directly from the members’ own experience with the stress of registering for classes each semester, which led the group to investigate whether graph coloring algorithms could be applied to course scheduling. The team implemented a greedy graph-coloring heuristic in Python using the NetworkX library, applied it

to a small hand-constructed example of eight students across seven courses, and validated the result both by an explicit adjacency check and by comparison against NetworkX’s built-in `greedy_color` function.

This paper is written as a retrospective, honest documentation of that coursework artifact. It is not a new research contribution: no novel algorithm is proposed, and no large-scale or real-world scheduling dataset was used. All facts, figures, and limitations reported here are drawn directly from the project’s README and the team’s original written report and presentation slides.

II. RELATED WORK

Graph coloring has long been used as a natural model for scheduling and resource-allocation problems in which some pairs of tasks cannot be assigned the same resource. In examination and course timetabling, courses or exams are modeled as vertices and a conflict edge is placed between two courses whenever they share at least one student, so that a proper vertex coloring with k colors corresponds to a timetable using k time slots with no student conflicts. Because determining the chromatic number of a general graph is NP-complete, exact optimal colorings are computationally intractable for large timetabling instances, and practical systems instead rely on heuristics.

Commonly used heuristics include greedy sequential coloring, in which vertices are colored in some fixed order and each is assigned the smallest color not already used by its colored neighbors; degree-ordered variants such as the Welsh–Powell heuristic, which color vertices in decreasing order of degree in an attempt to reduce the total number of colors used; and saturation-degree heuristics such as DSATUR, which prioritize vertices with the most distinctly colored neighbors. More sophisticated approaches to real-world timetabling additionally incorporate constraint programming, simulated annealing, and genetic algorithms to handle soft constraints such as instructor and room preferences. The team’s own bibliography, reproduced in the project’s README, cites prior work applying graph-coloring approaches to course timetabling (Ganguli & Roy, 2017) as well as standard discrete-mathematics treatments of graph coloring (Rosen, 2012) and practitioner tutorials on implementing graph coloring with NetworkX (Boutellier, 2021). This project sits at the introductory end of that

spectrum: it demonstrates the basic greedy-coloring approach on a small illustrative example rather than attempting to reproduce or extend a published algorithm on real institutional data.

III. PROBLEM FORMULATION

Let $C = \{c_1, c_2, \dots, c_n\}$ be the set of courses to be scheduled, and let $S = \{s_1, s_2, \dots, s_m\}$ be the set of students, where each student s_i is enrolled in some subset of courses $\text{enroll}(s_i) \subseteq C$. The conflict graph $G = (V, E)$ is defined as:

- $V = C$: each course is a vertex.
- $(c_i, c_j) \in E$ if and only if there exists at least one student $s_k \in S$ such that $\{c_i, c_j\} \subseteq \text{enroll}(s_k)$, i.e., some student is enrolled in both courses.

A time-slot assignment is a coloring function $\text{color} : V \rightarrow \{0, 1, \dots, k-1\}$ that maps each course to one of k time slots. The assignment is *valid* (conflict-free) if and only if it is a proper coloring of G :

$$\forall (u, v) \in E, \quad \text{color}(u) \neq \text{color}(v). \quad (1)$$

The scheduling optimization goal is to find a valid coloring that minimizes k , the number of colors (time slots) used. The smallest such k is the chromatic number $\chi(G)$ of the conflict graph. Because computing $\chi(G)$ exactly is NP-complete in general, the project uses a greedy heuristic that produces a valid, but not necessarily minimum, coloring.

IV. METHODOLOGY

A. Greedy Coloring (Implemented)

The team implemented and used a standard greedy sequential vertex-coloring algorithm, applied in whatever order NetworkX’s default vertex iteration produces for the graph. For each vertex, the algorithm examines the colors already assigned to its already-colored neighbors and assigns the smallest non-negative integer color not present among them. The implementation, taken directly from the project’s `main.py`, is:

```
def is_valid_coloring(graph, coloring):
    for u, v in graph.edges():
        if coloring[u] == coloring[v]:
            return False
    return True

def greedy_coloring(graph):
    coloring = {}
    for node in graph.nodes():
        neighbor_colors = {
            coloring[n] for n in graph.neighbors(node)
            if n in coloring
        }
        available_colors = set(
            range(len(neighbor_colors) + 1)
        ) - neighbor_colors
        coloring[node] = min(available_colors)
    return coloring
```

`is_valid_coloring` checks the coloring’s correctness directly from the problem formulation: it iterates over every edge and fails if any pair of adjacent vertices shares a color.

`greedy_coloring` performs a single pass over the vertices, so its running time is linear in the number of vertices and edges examined for neighbor colors.

The team’s result was additionally cross-checked against NetworkX’s own built-in greedy coloring routine, `networkx.greedy_color`, which implements the same class of algorithm (greedy coloring under a specified or default vertex ordering strategy).

B. Welsh–Powell Heuristic (Discussed, Not Implemented)

The Welsh–Powell heuristic improves on plain greedy coloring by first sorting vertices in decreasing order of degree before applying the same greedy color-assignment rule, on the intuition that coloring highly connected vertices first tends to reduce the total number of colors needed. According to the team’s own project report and the README’s documented tradeoffs, the team explicitly discussed degree-based vertex ordering (naming it in the context of “prioritizing nodes with the most connections first”) as a possible improvement and future-work item, but the team’s submitted implementation does not perform this reordering and colors vertices in NetworkX’s default iteration order instead. We report this heuristic here as part of the methodology considered by the team, and we are explicit that it is not part of the code that was run to produce the results in Section V.

V. EXPERIMENTAL SETUP

The evaluation dataset is a single, small, hand-constructed example built by the team to illustrate the algorithm, not a real institutional scheduling dataset. It consists of 8 students and 7 courses:

- Students: Emily, Benjamin, Sophia, Ethan, Olivia, Liam, Noah, Adam.
- Courses: Accounting, Coding, Drama, Economic, Food Science, Biology, Geology.

Each student’s course list induces conflict edges among the courses they take: for a student enrolled in three courses, all three pairwise edges among those courses are added; for a student enrolled in two courses, the single edge between them is added. This produces a conflict graph over the 7 course vertices. The implementation uses Python with the NetworkX library for graph construction and coloring, and Matplotlib for visualizing the resulting colored graph. The project includes a test suite (`tests/test_main.py`) of 13 unit tests, covering `is_valid_coloring` on two-node, triangle, and empty-graph cases; `greedy_coloring` on random Erdős–Rényi graphs, path graphs, triangles, K_4 , and isolated-vertex graphs; and the team’s 8-student/7-course example itself. According to the repository’s continuous-integration configuration, these tests run under a headless Matplotlib backend across Python 3.10, 3.11, and 3.12, and the README reports a runtime of approximately 3.5 seconds for the full suite.

Course Scheduling Example

Classes	Students	Time Slot
Accounting	Emily, Benjamin, Sophia	Monday
Biology	Sophia, Ethan	Tuesday
Coding	Olivia, Emily	Tuesday
Drama	Emily, Liam, Noah	Wednesday
Economic	Olivia, Benjamin, Ethan	Wednesday
Food Science	Adam, Liam, Benjamin	Tuesday
Geology	Adam, Noah	Monday

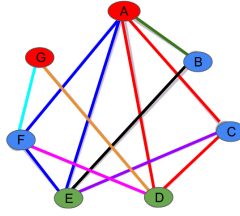


Fig. 1. The 7-course conflict graph, colored by the greedy vertex-coloring algorithm into 3 color classes (time slots), as produced by the team’s Matplotlib visualization.

VI. RESULTS

Fig. 1 shows the conflict graph produced by the team’s visualization script, with each course vertex colored according to the greedy algorithm’s output.

On the 8-student/7-course example, the greedy algorithm produced the following assignment of courses to time slots (colors 0, 1, 2, labeled Monday/Tuesday/Wednesday in the team’s presentation):

- Color 0 (Monday): Accounting, Geology
- Color 1 (Tuesday): Coding, Food Science, Biology
- Color 2 (Wednesday): Drama, Economic

This coloring uses $k = 3$ colors and was verified to be valid by `is_valid_coloring` (no adjacent courses share a color). The number of distinct colors used, computed as $|\{\text{color}(v) : v \in V\}|$, was 3. This matches the number of colors produced by NetworkX’s built-in `greedy_color` on the same graph, so on this particular small instance the team’s greedy implementation happened to agree with NetworkX’s own greedy coloring. The README states that all 13 unit tests pass, checking structural correctness (the no-adjacent-same-color invariant) and color counts on the test graphs described above, rather than asserting exact per-vertex color values or runtimes (which the team notes can vary with NetworkX version and Python dictionary iteration order).

No experiments were run at larger scale, and no comparison of the greedy heuristic against a Welsh–Powell or DSATUR implementation was performed, since neither of the latter was implemented in the codebase. We therefore report only the single-instance result above and do not claim any general conflict-reduction percentage, since none was measured.

VII. LIMITATIONS

The following limitations are drawn directly from the trade-offs section of the project’s README, which the authors state reflects the conclusions of the original written report:

- **Greedy coloring is a heuristic, not optimal.** Since graph coloring is NP-complete, greedy coloring finds a feasible coloring quickly but is not guaranteed to use the

minimum possible number of colors (the chromatic number). On the team’s 7-course example the greedy result happened to match NetworkX’s own greedy coloring, but on harder graphs the greedy result can require more colors than the optimum.

- **Sensitivity to vertex order.** The greedy algorithm’s output depends on the order in which vertices are processed. Degree-based orderings such as Welsh–Powell, or saturation-based orderings such as DSATUR, are known to often improve results, but the team explicitly chose not to implement either and instead named degree-based prioritization as future work.
- **Simulated, not real, data.** The 8-student/7-course example was constructed by hand to illustrate the algorithm. Real university scheduling involves hundreds of courses and additional constraints – faculty availability, classroom capacity, student soft preferences, and exam conflicts – none of which are modeled in this project.
- **No backtracking or metaheuristic search.** The team’s conclusion names backtracking algorithms and genetic algorithms as the next class of methods to explore, since both can find optimal or near-optimal colorings at higher computational cost, but neither was attempted in this project.

VIII. CONCLUSION

This paper documented a CS 5002 coursework project at Northeastern University in which a team of five students modeled course scheduling as a graph coloring problem and implemented a greedy vertex-coloring heuristic in Python using NetworkX. On a small, hand-built example of 8 students and 7 courses, the greedy implementation produced a valid conflict-free schedule using 3 time slots, matching the result of NetworkX’s built-in greedy coloring routine on the same graph. The project’s own documentation is explicit that this is a heuristic result on a toy instance rather than a demonstration of optimality or scalability, that a degree-ordered Welsh–Powell heuristic was discussed but not implemented, and that real institutional scheduling would require handling constraints – faculty availability, room capacity, and student preferences among others – that are outside this project’s scope. The value of the project lies in demonstrating, at small scale, how a well-known combinatorial abstraction (graph coloring) can be mapped onto a familiar real-world problem (course scheduling) and implemented with a standard graph library, which was the stated learning objective of the course assignment.

ACKNOWLEDGMENT

This project was completed as a team effort for CS 5002 (Discrete Structures) at Northeastern University, Spring 2024, by Yao Chong (Charles) Chow, Kaustubha Eluri, Yuneng (James) Li, Jonathan Mowat, and Nicholas Ung. The author thanks the team for their collaboration on the original implementation, written report, and presentation.

REFERENCES

- [1] R. Ganguli and S. Roy, "A Study on Course Timetable Scheduling using Graph Coloring Approach," *International Journal of Computational and Applied Mathematics*, vol. 12, no. 2, pp. 469–485, 2017.
- [2] K. H. Rosen, *Discrete Mathematics and Its Applications*, 7th ed. New York, NY, USA: McGraw-Hill, 2012, ch. 10.2 and 10.8.
- [3] Y. Boutellier, "Graph Coloring with networkx," *Towards Data Science*, 2021.